Programmatic Modeling of Concurrent Composition and High-Order Observers for Labeled Finite-State Automata

Zixi Wang

Contents

1 Regular Language and FAs **2**
1.1 Alphabets, words, and languages 2
1.2 Regular languages 2
1.3 Deterministic and nondeterministic finite automata 3

2 LFSAs **5**

3 Concurrent Composition **6**

4 Observer **7**

5 High-order observer **8**
5.1 Model of Agent 9
5.2 Pipeline to get the high-order observer 9

References **11**

1 Regular Language and FAs

The following definitions summarize the background needed in this project.

1.1 Alphabets, words, and languages

An **alphabet** is a finite, nonempty set of letters, usually denoted by Σ. A **word** over Σ is a finite sequence of letters from Σ. More precisely, a word of length n has the form

$$w = a_1 a_2 \cdots a_n \qquad a_i \in \Sigma.$$

The empty word is denoted by ϵ. For every $n \in \mathbb{N}$,

$$\Sigma^0 = \{\epsilon\}, \qquad \Sigma^n = \{a_1 a_2 \cdots a_n \mid a_i \in \Sigma,\ 1 \le i \le n\} \quad (n \ge 1).$$

The **Kleene star**

$$\Sigma^* = \bigcup_{n \ge 0} \Sigma^n = \{\epsilon\} \cup \Sigma \cup \Sigma^2 \cup \cdots$$

is the set of all finite words over Σ. A **formal language over** Σ is any set

$$L \subseteq \Sigma^*.$$

Given $L_1, L_2 \subseteq \Sigma^*$, there are several basic operations:

$$\begin{aligned} L_1 \cup L_2 &= \{w \in \Sigma^* \mid w \in L_1 \text{ or } w \in L_2\}, \\ L_1 \cap L_2 &= \{w \in \Sigma^* \mid w \in L_1 \text{ and } w \in L_2\}, \\ \overline{L_1} &= \Sigma^* \setminus L_1, \\ L_1 \setminus L_2 &= \{w \in L_1 \mid w \notin L_2\}, \\ L_1 L_2 &= \{uv \mid u \in L_1,\ v \in L_2\}. \end{aligned}$$

For a language L, define $L^0 = \{\epsilon\}$ and $L^{n+1} = L^n L$. Its **Kleene star** is

$$L^* = \bigcup_{n \ge 0} L^n.$$

1.2 Regular languages

A **regular expression over** Σ is defined recursively. The expressions $\emptyset$, ϵ, and a for every $a \in \Sigma$ are regular expressions. If r and s are regular expressions, then

$$(r), \qquad r \cup s, \qquad rs, \qquad r^*$$

are also regular expressions, representing parentheses, union, concatenation, and Kleene star, respectively.

The language denoted by a regular expression r, written $L(r)$, is defined recursively by

$$\begin{aligned} L(\emptyset) &= \emptyset, & L(\epsilon) &= \{\epsilon\}, & L(a) &= \{a\}, \\ L(r \cup s) &= L(r) \cup L(s), & L(rs) &= L(r)L(s), & L(r^*) &= L(r)^*. \end{aligned}$$

A language $L \subseteq \Sigma^*$ is **regular** if and only if there exists a regular expression r over Σ such that

$$L = L(r).$$

Equivalently, L is regular if and only if it is recognized by a finite automaton.

Regular languages are closed under union, intersection, complement, difference, concatenation, and Kleene star. In other words, if L_1 and L_2 are regular, then

$$L_1 \cup L_2, \quad L_1 \cap L_2, \quad \overline{L_1}, \quad L_1 \setminus L_2, \quad L_1 L_2, \quad L_1^*$$

are also regular.

Let a DFA $A = (Q, \Sigma, \delta, q_0, F)$ recognize the regular language L, and choose the **pumping length**

$$p = |Q|.$$

For every $w \in L$ with $|w| \geq p$, the run of A while reading the first p letters visits a state more than once. Therefore,

$$\forall w \in L \text{ with } |w| \geq p, \ \exists x, y, z \in \Sigma^*$$

such that

- $w = xyz$;
- $|xy| \leq p$;
- $y \neq \epsilon$;
- $\forall i \in \mathbb{N}$, $xy^i z \in L$.

Thus, the number of states of a DFA recognizing L is a valid pumping length, although it is not necessarily the smallest possible pumping length.

1.3 Deterministic and nondeterministic finite automata

A deterministic finite automaton (DFA) is a quintuple

$$A = (Q, \Sigma, \delta, q_0, F),$$

where:

- Q is a finite set of states;
- Σ is the input alphabet;
- $\delta : Q \times \Sigma \to Q$ is the transition function;
- $q_0 \in Q$ is the initial state;
- $F \subseteq Q$ is the set of final states.

The transition function extends from letters to words. For $q \in Q$, $w \in \Sigma^*$, and $a \in \Sigma$,

$$\delta(q, \epsilon) = q, \qquad \delta(q, wa) = \delta(\delta(q, w), a).$$

The DFA accepts w exactly when $\delta(q_0, w) \in F$. Its recognized language is

$$L(A) = \{w \in \Sigma^* \mid \delta(q_0, w) \in F\}.$$

A nondeterministic finite automaton (NFA) is a quintuple

$$N = (Q, \Sigma, \delta, Q_0, F),$$

where:

- Q is a finite set of states;
- Σ is the input alphabet;
- $\delta : Q \times \Sigma \to 2^Q$ is the transition function;
- $Q_0 \subseteq Q$ is the set of initial states;
- $F \subseteq Q$ is the set of final states.

Here, $2^Q = \{X \mid X \subseteq Q\}$ is the power set of Q, so $\delta(q, a)$ may contain zero, one, or several successor states. Extend δ to sets of states and words by

$$\delta(X, \epsilon) = X, \qquad \delta(X, wa) = \bigcup_{q \in \delta(X, w)} \delta(q, a),$$

for $X \subseteq Q$, $w \in \Sigma^*$, and $a \in \Sigma$. The NFA accepts a word w if at least one state reachable from an initial state is accepting:

$$\delta(Q_0, w) \cap F \neq \emptyset.$$

Therefore, the language recognized by N is

$$L(N) = \{w \in \Sigma^* \mid \delta(Q_0, w) \cap F \neq \emptyset\}.$$

Every NFA can be converted into an equivalent DFA by the **subset construction**.

Kleene's theorem establishes the equivalence

$$\text{regular expressions} \iff \text{NFAs} \iff \text{DFAs}.$$

In this project, the underlying system is usually written as

$$G = (Q, E, \delta, Q_0),$$

where E is an event alphabet, $\delta \subseteq Q \times E \times Q$ is a transition relation, and $Q_0 \subseteq Q$ is a set of initial states. Accepting states are omitted because the project studies generated runs and state estimates rather than language acceptance. This transition-system form may be nondeterministic and may have multiple initial states. Adding an observation alphabet and a labeling function to G produces the LFSA introduced in the next section.

2 LFSAs

According to Definition 1 in Zhang (2023), a **labeled finite-state automaton (LFSA)** is a sextuple

$$\mathcal{S} = (Q, E, \delta, Q_0, \Sigma, \ell),$$

where:

- Q is a finite set of states;
- E is a finite alphabet of events;
- $\delta : Q \times E \to 2^Q$ is the transition function;
- $Q_0 \subseteq Q$ is the set of initial states;
- Σ is a finite alphabet of observable labels;
- $\ell : E \to \Sigma \cup \{\epsilon\}$ is the labeling function.

A transition $q' \in \delta(q, e)$, equivalently $(q, e, q') \in \delta$, means that the occurrence of event e moves the LFSA from state q to state q'. Since $\delta(q, e)$ is a set of states, an LFSA may be nondeterministic.

The event alphabet is partitioned into the observable and unobservable event sets

$$E_o = \{e \in E \mid \ell(e) \in \Sigma\}, \qquad E_{uo} = \{e \in E \mid \ell(e) = \epsilon\},$$

When an event in E_o occurs, its label is observed. When an event in E_{uo} occurs, the observer receives no output.

The transition function is extended from events to finite event sequences. For $q \in Q$, $s \in E^*$, and $e \in E$,

$$\delta(q, \epsilon) = \{q\}, \qquad \delta(q, se) = \bigcup_{p \in \delta(q,s)} \delta(p, e).$$

For $X \subseteq Q$, define

$$\delta(X, s) = \bigcup_{q \in X} \delta(q, s).$$

The labeling function is extended from events to event sequences by:

$$\ell(\epsilon) = \epsilon, \qquad \ell(e_1 e_2 \cdots e_n) = \ell(e_1)\ell(e_2)\cdots\ell(e_n).$$

The set of finite event traces generated by $\mathcal{S}$ is

$$L(\mathcal{S}) = \{s \in E^* \mid \delta(Q_0, s) \neq \emptyset\}.$$

The observable language generated by $\mathcal{S}$ is

$$\mathcal{L}(\mathcal{S}) = \{\ell(s) \in \Sigma^* \mid s \in L(\mathcal{S})\}.$$

For an observed word $\sigma \in \Sigma^*$, the **current-state estimate** is

$$\mathcal{M}(\mathcal{S}, \sigma) = \{q \in Q \mid \exists q_0 \in Q_0,\ \exists s \in E^* : \ell(s) = \sigma \text{ and } q \in \delta(q_0, s)\}.$$

Thus, an LFSA distinguishes an internal event sequence s from its observable label sequence $\ell(s)$. Different event sequences may generate the same observed word, which is the source of state uncertainty studied by concurrent composition and observers.

3 Concurrent Composition

According to Definition 2 in Zhang (2023), consider two LFSAs with the same output alphabet Σ:

$$\mathcal{S}_i = (Q_i, E_i, \delta_i, Q_{0i}, \Sigma, \ell_i), \qquad i \in \{1, 2\}.$$

For each $\mathcal{S}_i$, define

$$E_{io} = \{e \in E_i \mid \ell_i(e) \in \Sigma\}, \qquad E_{iuo} = \{e \in E_i \mid \ell_i(e) = \epsilon\}.$$

Their **concurrent composition** is the LFSA

$$\mathrm{CC}_A(\mathcal{S}_1, \mathcal{S}_2) = (Q', E', \delta', Q'_0, \Sigma, \ell'),$$

whose components are defined as follows.

1. The state set is the Cartesian product

$$Q' = Q_1 \times Q_2.$$

   Thus, a state (q_1, q_2) records the states currently occupied by the two component LFSAs.

2. The initial-state set is

$$Q'_0 = Q_{01} \times Q_{02}.$$

3. The observable events synchronize events having the same label:

$$E'_o = \{(e_1, e_2) \in E_{1o} \times E_{2o} \mid \ell_1(e_1) = \ell_2(e_2)\}.$$

4. The unobservable events occur independently in either component:

$$E'_{uo} = \{(e_1, \epsilon) \mid e_1 \in E_{1uo}\} \cup \{(\epsilon, e_2) \mid e_2 \in E_{2uo}\}.$$

   Therefore,

$$E' = E'_o \cup E'_{uo}.$$

5. For a synchronized observable event $(e_1, e_2) \in E'_o$,

$$((q_1, q_2), (e_1, e_2), (q'_1, q'_2)) \in \delta' \quad \Longleftrightarrow \quad (q_1, e_1, q'_1) \in \delta_1 \text{ and } (q_2, e_2, q'_2) \in \delta_2.$$

6. For a left-component unobservable event $(e_1, \epsilon) \in E'_{uo}$,

$$((q_1, q_2), (e_1, \epsilon), (q'_1, q'_2)) \in \delta' \quad \Longleftrightarrow \quad (q_1, e_1, q'_1) \in \delta_1 \text{ and } q_2 = q'_2.$$

7. For a right-component unobservable event $(\epsilon, e_2) \in E'_{uo}$,

$$((q_1, q_2), (\epsilon, e_2), (q'_1, q'_2)) \in \delta' \quad \Longleftrightarrow \quad q_1 = q'_1 \text{ and } (q_2, e_2, q'_2) \in \delta_2.$$

8. The labeling function is

$$\ell'((e_1, e_2)) = \ell_1(e_1) = \ell_2(e_2),$$

where the labeling functions are extended by $\ell_1(\epsilon) = \ell_2(\epsilon) = \epsilon$.

Consequently, observable transitions can occur together only when they produce the same label, whereas unobservable transitions can occur separately without changing the other component's state. The concurrent composition therefore collects pairs of runs from $\mathcal{S}_1$ and $\mathcal{S}_2$ that generate the same observable label sequence.

If $\mathcal{S}_1 = \mathcal{S}_2 = \mathcal{S}$, then

$$\mathrm{CC}_A(\mathcal{S}, \mathcal{S}) = \mathrm{CC}_A(\mathcal{S})$$

is called the **self-composition** of $\mathcal{S}$.

4 Observer

According to Definition 3 in Zhang (2023), consider an LFSA

$$\mathcal{S} = (Q, E, \delta, Q_0, \Sigma, \ell)$$

with observable event set E_o and unobservable event set E_{uo}. Its **observer** is the deterministic finite automaton

$$\mathcal{S}_{\mathrm{obs}} = (Q_{\mathrm{obs}}, \Sigma_{\mathrm{obs}}, \delta_{\mathrm{obs}}, q_{0\ \mathrm{obs}}).$$

Each observer state is a set of possible states of the original LFSA. The components of the observer are defined as follows.

1. The observer state set is the power set

$$Q_{\mathrm{obs}} = 2^Q.$$

2. The observer alphabet is the set of observable labels generated by events:

$$\Sigma_{\mathrm{obs}} = \ell(E_o) = \ell(E) \setminus \{\epsilon\}.$$

3. For any $X \subseteq Q$, its **unobservable reach** is

$$\mathrm{UR}(X) = \bigcup_{q \in X} \bigcup_{s \in E_{uo}^*} \delta(q, s).$$

4. The initial observer state contains every state reachable from an original initial state through only unobservable events:

$$q_{0\ \mathrm{obs}} = \mathrm{UR}(Q_0).$$

5. For $X \in Q_{\mathrm{obs}}$ and $a \in \Sigma_{\mathrm{obs}}$, first take every observable event labeled a, and then take the unobservable reach:

$$\delta_{\mathrm{obs}}(X, a) = \mathrm{UR}\left(\bigcup_{q \in X} \bigcup_{\substack{e \in E_o \\ \ell(e) = a}} \delta(q, e) \right).$$

Because $\delta_{\mathrm{obs}}(X, a)$ is one uniquely determined subset of Q, the observer is deterministic even when the original LFSA is nondeterministic. For every observed word $\sigma \in \Sigma_{\mathrm{obs}}^*$,

$$\delta_{\mathrm{obs}}(q_{0\ \mathrm{obs}}, \sigma) = \mathcal{M}(\mathcal{S}, \sigma).$$

Thus, after reading σ, the observer state is exactly the current-state estimate of the LFSA: it contains all and only the states in which $\mathcal{S}$ could be after producing σ. Although $Q_{\mathrm{obs}} = 2^Q$ contains all subsets theoretically, an implementation usually constructs only the subsets reachable from $q_{0\ \mathrm{obs}}$.

5 High-order observer

Consider an FSA

$$G = (Q, E, \delta, Q_0)$$

and an ordered sequence of agents $A_1, A_2, \ldots, A_n$. Agent A_i observes an event trace $s \in L(G)$ through its labeling function

$$\ell_i : E \to \Sigma_i \cup \{\epsilon\}.$$

An ordinary observer gives an agent's estimate of the current state of G. This is an **order-1 estimate**:

$$\mathcal{M}_{A_1}(G, \alpha) = \{q \in Q \mid \exists q_0 \in Q_0,\ \exists s \in L(G) : \ell_1(s) = \alpha \text{ and } q \in \delta(q_0, s)\} \in 2^Q.$$

A high-order estimate describes one agent's estimate of another agent's estimate. For example, after A_2 observes $\beta \in \ell_2(L(G))$, its order-2 estimate of what A_1 may know is

$$\mathcal{M}_{A_1 \leftarrow A_2}(G, \beta) = \{\mathcal{M}_{A_1}(G, \ell_1(s)) \mid s \in L(G),\ \ell_2(s) = \beta\} \in 2^{2^Q}.$$

Thus, A_1 estimates the current state of G, whereas A_2 estimates the possible state estimates held by A_1. The arrow in $A_1 \leftarrow A_2$ indicates that the final inference is made by A_2 about A_1.

5.1 Model of Agent

Consider an FSA

$$G = (Q, E, \delta, Q_0).$$

An **agent** A_i is modeled by the triple

$$A_i = (E_i, \Sigma_i, \ell_i),$$

where:

- $E_i \subseteq E$ is the set of events observable to A_i;
- Σ_i is the finite alphabet of labels observed by A_i;
- $\ell_i : E_i \to \Sigma_i$ is the labeling function of A_i.

The labeling function is extended to $\bar{\ell}_i : E \to \Sigma_i \cup \{\epsilon\}$ by

$$\bar{\ell}_i(e) = \begin{cases} \ell_i(e), & e \in E_i, \\ \epsilon, & e \in E \setminus E_i. \end{cases}$$

For simplicity, the extended function $\bar{\ell}_i$ is also denoted by ℓ_i in the following. Thus, an event in E_i produces an observable label, while an event outside E_i is unobservable to A_i. Different observable events may have the same label.

For an event trace $s = e_1 e_2 \cdots e_n \in E^*$, the observation made by A_i is

$$\ell_i(s) = \ell_i(e_1)\ell_i(e_2)\cdots\ell_i(e_n) \in \Sigma_i^*,$$

where every occurrence of ϵ disappears under concatenation. Two event traces are indistinguishable to A_i if they produce the same observation:

$$s \sim_{A_i} t \quad \Longleftrightarrow \quad \ell_i(s) = \ell_i(t).$$

Combining the FSA G with the observation mechanism of A_i produces the LFSA

$$G_{A_i} = (Q, E, \delta, Q_0, \Sigma_i, \ell_i).$$

The agent does not change the states, events, or transitions of G; it only determines how the events are observed. Consequently, different agents may produce different LFSAs from the same FSA because they may have different observable event sets, alphabets, or labeling functions.

5.2 Pipeline to get the high-order observer

Consider an FSA $G = (Q, E, \delta, Q_0)$ and two ordered agents $A_1 = (E_1, \Sigma_1, \ell_1)$ and $A_2 = (E_2, \Sigma_2, \ell_2)$. An event outside E_i is unobservable to A_i, so its label under ℓ_i is ϵ. The order-2 observer is constructed as follows.

1. Combine G with A_1's observation mechanism to obtain the LFSA

   $$G_{A_1} = (Q, E, \delta, Q_0, \Sigma_1, \ell_1).$$

   In the program, `src/fa_agent_to_lfsa.py` reads the FSA JSON and the A_1 JSON and writes the LFSA JSON.

2. Apply the powerset construction to G_{A_1}:

$$\mathrm{Obs}_{\ell_1}(G) =: \mathrm{Obs}_{A_1}\,.$$

Each observer state $X \subseteq Q$ is A_1's current-state estimate after an observation. The program `src/observer.py` constructs the nonempty reachable part of this observer. Although the observer is mathematically a DFA, its JSON representation uses identity labels, $\ell_{\mathrm{obs}}(a) = a$, so it can be passed to the concurrent-composition program.

3. Concurrently compose G_{A_1} and Obs_{A_1}:

$$CC_{\ell_1}\left(G_{A_1}, \mathrm{Obs}_{A_1}\right).$$

A state of this composition has the form (q, X), where q is the actual state of G and X is the state estimate held by A_1. Observable transitions synchronize under the same ℓ_1-label, while unobservable transitions of G_{A_1} interleave. This step is implemented by `src/CC.py`.

4. The composition is still labeled from A_1's perspective. To prepare it for A_2, first project every paired composition event

$$(e_1, e_2) \longmapsto e_2$$

and remove the existing ℓ_1-labels. This produces the underlying FSA whose states remain pairs (q, X). The operation is provided by `src/lfsa_to_fa.py` using its `-project-left` option.

5. Combine this projected FSA with A_2. This replaces ℓ_1 with ℓ_2 and produces the LFSA $CC^{G,\mathrm{Obs}}_{A_1 \to A_2}$. In the program, run `src/fa_agent_to_lfsa.py` with the projected FSA and the A_2 agent JSON.

6. Apply `src/observer.py` to the A_2-labeled composition:

$$\mathrm{Obs}_{\ell_2}\left(CC^{G,\mathrm{Obs}}_{A_1 \to A_2}\right) =: \mathrm{Obs}_{A_1 \leftarrow A_2}(G).$$

This is the order-2 observer. A state has the form

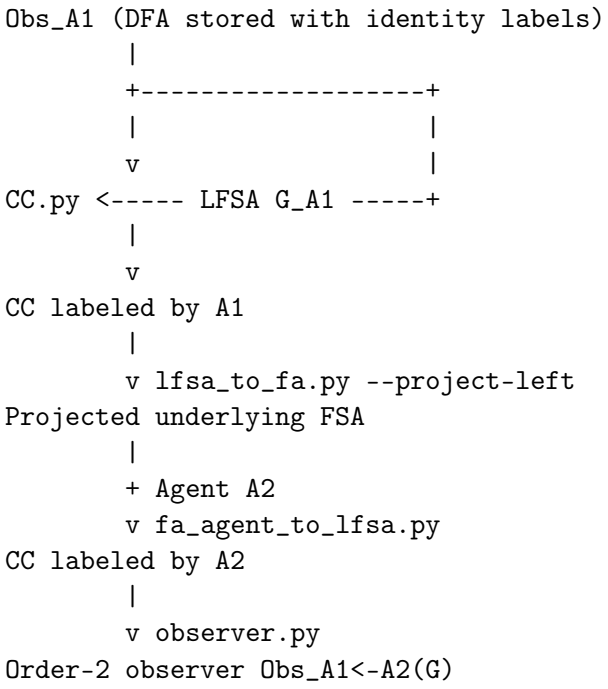
$$\{(q_1, X_1), \ldots, (q_n, X_n)\}.$$

After A_2 observes a word α, its possible inferences of A_1's estimate are

$$M_{A_1 \leftarrow A_2}(G, \alpha) = \{X_1, \ldots, X_n\}.$$

The programmatic data flow is therefore

```
FSA G + Agent A1
        |
        v
LFSA G_A1
        |
        v observer.py
```



For $n > 2$, the construction is repeated recursively: project the current composition onto the original event component, apply the next agent's labeling function, and construct another observer. Each new observer adds one level of state-estimate nesting. In the general case, constructing an order-n observer has n-EXPTIME complexity.

References

[1] K. Zhang, "A unified concurrent-composition method to state/event inference and concealment in labeled finite-state automata as discrete-event systems," *Annual Reviews in Control*, vol. 56, article 100902, 2023. https://doi.org/10.1016/j.arcontrol.2023.100902.

[2] K. Zhang, X. Han, A. Giua, and C. Seatzu, "[Draft] High-order observers and high-order state-estimation-based properties of discrete-event systems," *arXiv preprint* arXiv:2408.06141, 2024. https://doi.org/10.48550/arXiv.2408.06141.